

S istemas Operativos

1º Parcial 1C2026 TM – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

- 1- a) Grado de multiprogramación
b) Dos cambios de modo
c) SJF y/o SRT
d) Mutua exclusión, Retención y Espera, Sin Desalojo, Espera Circular

- 2- i) b- Starvation
ii) a- Proceso en ejecución finaliza
iii) b- No permiten paralelismo entre hilos pares
iv) c- Se utiliza en la estrategia de evasión de deadlocks

3- Por cada petición se levanta un ULT ya que **“se desea poder planificar la atención de peticiones bajo un algoritmo propietario (independiente del SO)”**. De esta manera, cada petición será planificada bajo un algoritmo definido por la biblioteca de ULTs del proceso.

Luego **“se desea que algún error fatal en la atención de una petición no comprometa por completo al sistema”**, por lo que necesitaremos al menos 2 Procesos en el sistema destinados atender estas peticiones, de esta manera, cada proceso hace uso de su espacio de memoria de forma totalmente independiente, imposibilitando que un error fatal afecte al resto de Procesos.

Por último **“se busca que el servidor tenga el mayor paralelismo posible en la atención de peticiones”**, para cumplir esto podríamos tener 32 Procesos en el sistema para atender peticiones en lugar de 2, otra opción es que cada Proceso tenga a su vez múltiples KLTs para aprovechar los 32 Procesadores.

4- Cumplen con los requisitos:

- **Mutua exclusión:** sólo un proceso puede adquirir el semáforo a la vez, el resto se quedarán bloqueados.
- **Progreso:** los únicos que pueden evitar que otro proceso adquiriera el semáforo son los que también están realizando el wait.
- **Espera limitada:** los procesos se bloquean en una cola de dicho semáforo y luego se “despiertan” a medida que se hace signal al semáforo, respetando el orden de bloqueo.
- **Velocidad de los procesos:** la solución funciona de forma adecuada sin importar la velocidad de uso por parte de los procesos.

Una solución que implemente un **sistema de turnos** solamente cumpliría con Mutua Exclusión, sin embargo, no cumpliría con Progreso, ya que si es turno de un proceso, no permitiría entrar a otros procesos a la sección crítica aun si el mismo no está accediendo a la sección crítica.

5- La inversión de prioridades se da cuando, en un sistema con planificación de corto plazo por prioridades, un **Proceso más prioritario queda bloqueado** prolongadamente debido a estar esperando por un **recurso retenido por un proceso menos prioritario**.

Este problema puede solucionarse mediante **Herencia de Prioridades**.

Ejemplo de traza de ejecución:

Procesos P1, P2 y P3 cuyas prioridades son: $P1 < P2 < P3$

$t=0 \rightarrow$ P1 adquiere un recurso R $\rightarrow$ WAIT(semR)

$t=1 \rightarrow$ P3 ingresa al sistema y necesita el recurso R $\rightarrow$ WAIT(semR) se bloquea en la espera y sigue ejecutando P1.

$t=2 \rightarrow$ P2 ingresa al sistema y desaloja a P1 ya que tiene mayor prioridad, pero al no necesitar el recurso R continúa ejecutando.

En este ejemplo, P3 no puede ejecutar a pesar de ser más prioritario que P2 quien está en ejecución.

Práctica

1. a) generar=1, valorGenerado=0, acumulado=0, mutexAcum=1
b) Sí, se puede sacar mutexAcum
2. a) KC
b) En el instante t=15
c) SJF sin desalojo
d) HRRN

3.

P1			IO					F				
P2												F
P3									IO		F	
	0	1	2	3	4	5	6	7	8	9	10	11

4.

```
mutex_prompts = 1
hay_prompts = 0
mutex_repositorio = 1
hay_prs = 0
mutex_issues = 1
sem_issues[10] = {0,...,0}
```

Pedro	Agente Claudio Code (10 instancias)	Code Quality Bot (1 instancia)
<pre>while(1) { revisar_figma() prompt = generar_prompt() wait(mutex_prompts) agregar_prompt(prompts, prompt) signal(mutex_prompts) signal(hay_prompts) }</pre>	<pre>while(1) { wait(hay_prompts) wait(mutex_prompts) prompt = obtener_prompt(prompts) signal(mutex_prompts) pr = generar_pr(prompt) wait(mutex_repositorio) pushear_cambios(repositorio, pr) signal(mutex_repositorio) signal(hay_prs) wait(sem_issues[id0]) wait(mutex_issues) issue = obtenerIssue(issues) signal(mutex_issues) corregir_y_mergear_pr(pr, issue) }</pre>	<pre>while(1) { wait(hay_prs) wait(mutex_repositorio) pr = obtener_pr(repositorio) signal(mutex_repositorio) issue = revisar_codigo(pr) wait(mutex_issues) postear_issue(issues, issue) signal(mutex_issues) signal(sem_issues[id_agente(pr)]) }</pre>

5. a) No podría generarse deadlock entre los procesos 2 y 3 ya que **piden los recursos en un mismo orden**, por lo tanto, nunca se generaría espera circular que es condición necesaria para la ocurrencia de Deadlock.
b) El SO podría estar implementando la **estrategia de evasión**, particularmente si el recurso A ya estuviera tomado, sería prudente bloquear al proceso 1 al pedir el recurso B ya que su asignación podría causar deadlock si luego éste mismo pidiera el recurso A.